

Enlarging Hash Tables

Day 5 solution walkthrough

Competitive Programming Bootcamp

Statement excerpts:

- ▶ *“smallest prime number larger than $2n$ ”*
- ▶ *“if n happens not to be prime”*

Programming interpretation:

- ▶ For each input n , search upward from $2n + 1$ until a prime is found.
- ▶ Also test whether the old table size n is prime.
- ▶ The output changes only by adding a parenthetical warning for composite n .

Textbook connection: CP4 5.3.1 Prime Numbers

Core technique: Primality testing by trial division

- ▶ For each n , find the smallest prime strictly greater than $2n$.
- ▶ Also report if the original n is not prime.
- ▶ Input ends at $n = 0$.

- ▶ A candidate larger than $2n$ can be tested independently for primality.
- ▶ After 2, only odd candidates need to be checked.
- ▶ Trial division up to $\sqrt{n}$ is enough for the input scale.

- ▶ Implement `is_prime(n)`.
- ▶ Reject $n < 2$, accept $n = 2$, reject other even numbers.
- ▶ Try odd divisors from 3 through `isqrt(n)`.
- ▶ For each input n , start $\text{candidate} = 2n + 1$ and advance by 2 until prime.
- ▶ Print candidate, adding the warning if n itself is not prime.

- ▶ If n has a divisor above $\sqrt{n}$, the paired divisor is below $\sqrt{n}$.
- ▶ $2n + 1$ is always odd, so candidate $+= 2$ preserves odd candidates.
- ▶ The original n and enlarged candidate are separate primality checks.

- ▶ `is_prime` tests all possible factor pairs by checking divisors up to the square root.
- ▶ The candidate loop checks increasing integers greater than $2n$, skipping only even composites.
- ▶ The first prime found is therefore the smallest valid enlarged table size.

Complexity: $O(\sqrt{C})$ per primality test, where C is the candidate value; $O(1)$ memory.

- ▶ Use `math.isqrt` to avoid floating-point square-root issues.
- ▶ Remember the exact output format for non-prime original n .

Source lines: enlarginghashtables.py:43-70

```
import sys
from math import isqrt

def is_prime(n):
    if n < 2:
        return False

    if n == 2:
        return True

    if n % 2 == 0:
        return False

    limit = isqrt(n)

    for divisor in range(3, limit + 1, 2):
        if n % divisor == 0:
            return False

    return True
```

What this chunk does

- ▶ Numbers below 2 and even numbers above 2 are handled immediately.
- ▶ Only odd divisors up to $\text{sqrt}(n)$ need to be tested.
- ▶ If no divisor is found, n is prime.

Source lines: enlarginghashtables.py:73-85

```
def main():
    input = sys.stdin.buffer.readline

    while True:
        n = int(input())

        if n == 0:
            break

        candidate = 2 * n + 1
```

What this chunk does

- ▶ Input is processed until the sentinel value 0.
- ▶ The first possible new size is strictly greater than twice n .
- ▶ $2n + 1$ is odd, so later candidates can advance by 2.

Source lines: enlarginghashtables.py:87-101

```
while not is_prime(candidate):
    candidate += 2

if is_prime(n):
    print(candidate)

else:
    print(f"{candidate} ({n} is not prime)")

main()
```

What this chunk does

- ▶ The loop stops at the first prime candidate, making it the smallest valid new size.
- ▶ The original `n` is tested separately.
- ▶ Composite original sizes receive the required note.

Solution file: `practice_contests/day_5/enlarginghashtables.py`

Problem: <https://open.kattis.com/problems/enlarginghashtables>

Use this deck with the source code open beside it. The slides explain the reasoning; the code shows the exact input handling and output formatting.